SOFTWARE BY ADINO.T





Software Engineering Tutorial

Software Engineering is the systematic process of designing, developing, testing, and maintaining software to create reliable, efficient, and high-quality applications.

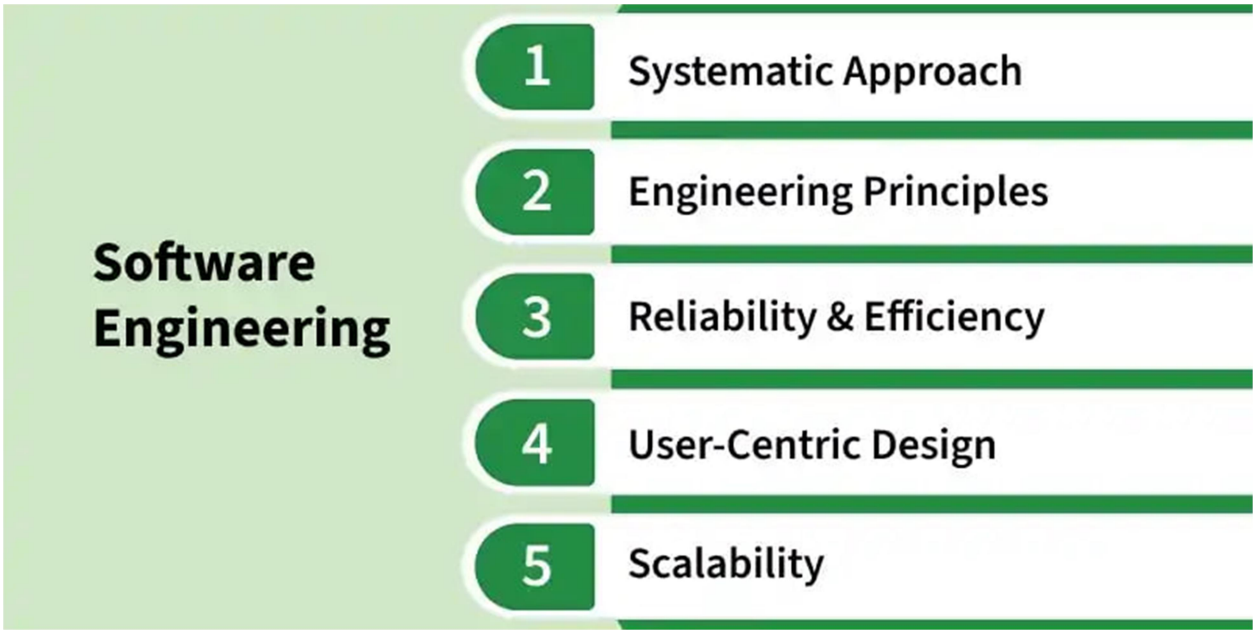
- It applies engineering principles, methods, and tools to develop software in a systematic manner.
- It supports teamwork, planning, documentation, testing, and maintenance to ensure successful software development throughout its lifecycle.
- It helps create software that meets user requirements while ensuring reliability, security, efficiency, and maintainability.

Introduction to Software Engineering

Software is a program or set of programs containing instructions that provide the desired functionality.

Engineering is the process of designing and building something that serves a purpose efficiently and cost-effectively.

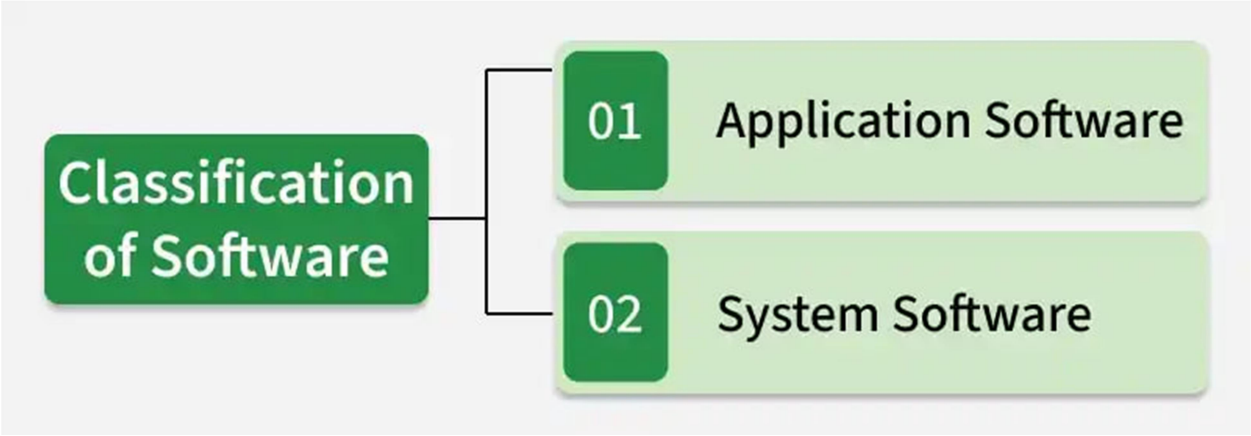
- Uses requirements analysis, design, testing, and maintenance techniques.
- Follows a well-defined software development lifecycle (SDLC).
- Improves development efficiency, cost management, and timely delivery.
- Adapts to new tools and technologies for complex software development.



Software and its Types

In a computer system, the software is basically a set of instructions or commands that tell a computer to do a specific task that serves its users.

- A software is not a physical thing like hardware, it rather makes the hardware work as per user requirements by giving instructions.
- Software enables hardware to perform tasks efficiently and helps users solve specific problems.



Classification of Software

1. System Software

System software is software that directly operates the computer hardware and provides the basic functionality to the users as well as to the other software to operate smoothly.

- System software basically controls a computer's internal functioning and also controls hardware devices such as monitors, printers, and storage devices, etc.
- It is like an interface between hardware and user applications, it helps them to communicate with each other because hardware understands machine language(i.e. 1 or 0) whereas user applications are work in human-readable languages like English, Hindi, German, etc.

Features

- Closer to the computer system.
- Written in a low-level language in general.
- Difficult to design and understand.
- Fast in speed(working speed).
- Less interactive for the users in comparison to application software.

Types of System Software

1. **Operating System:** It is the main program of a computer system. When the computer system is turned on, the operating system is the first software loaded into memory. Basically, it manages all the resources such as computer memory, CPU, printer, hard disk, etc., and provides an interface to the user, which helps the user to interact with the computer system.
2. **Programming Language Processor:** As we know that system software converts the human-readable language into a machine language and vice versa. So, the conversion is done by the language processor. It converts programs written in high-level programming languages like Java, C, C++, Python, etc (known as source code), into sets of instructions that are easily readable by machines(known as object code or machine code).
3. **Device Driver:** A device driver is a program or software that controls a device and helps that device to perform its functions. It enables the operating system to communicate with and control external hardware devices connected to the computer. **Example:** Printer, mouse, modem, etc.
4. **Utility Software:** This type of system software is used to support the computer infrastructure. Utility software analyzes, configures, optimizes, and maintains the computer system to ensure efficient performance. **Example:** Antivirus, disk defragmenter, memory tester, disk repair, disk cleaners, registry cleaners, disk space analyzer, etc.

2. Application Software

Software that performs special functions or provides functions that are much more than the basic operation of the computer is known as application software.

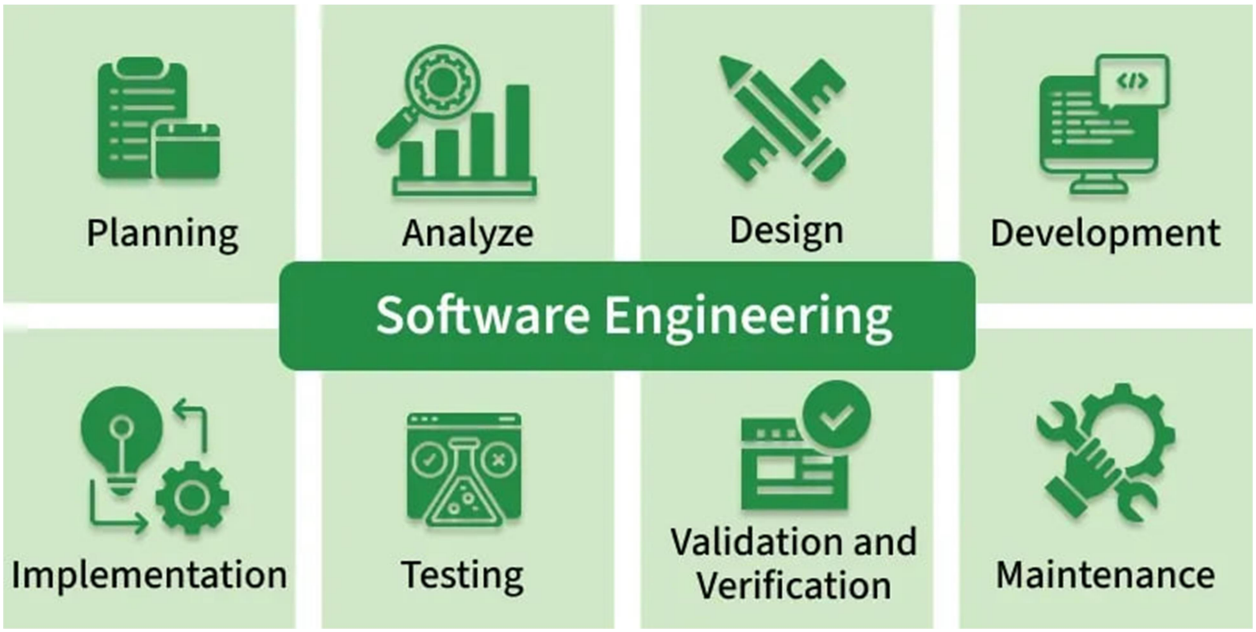
- Application software is designed to perform a specific task for end-users. It is a product or a program that is designed only to fulfill end-users' requirements.
- Examples include word processors, spreadsheets, database management, inventory, payroll programs, etc.

Features

- Perform more specialized tasks like word processing, spreadsheets, email, etc.
- Mostly, the size of the software is big, so it requires more storage space.
- More interactive for the users, so it is easy to use and design.
- Easy to design and understand.
- Written in a high-level language in general.

Types of Application Software

1. **General Purpose Software:** This type of application software is used for a variety of tasks and it is not limited to performing a specific task only. For example, MS-Word, MS-Excel, PowerPoint, etc.
2. **Customized Software:** This type of application software is used or designed to perform specific tasks or functions or designed for specific organizations. For example, railway reservation system, airline reservation system, invoice management system, etc.



Need of Software Engineering

Software engineering is essential for developing high-quality software in a structured, efficient, and scalable way, as it helps address challenges related to system complexity, cost, time management, and reliability.



1. Effectiveness

- Ensures software is developed according to defined standards and best practices
- Helps in achieving desired results efficiently by following a structured and disciplined development process

2. Handling Large Projects

- Large software systems involve multiple modules and teams
- Software engineering provides a structured way to manage such projects efficiently

3. Reliability

- Ensures software works correctly and meets user requirements
- Testing at every stage helps in identifying and fixing defects early

4. Time Management

- Proper planning avoids unnecessary work and reduces development cost
- Defined processes help in completing projects within deadlines

5. Improved Productivity

- Use of tools, automation, and proper planning increases efficiency
- Teams can deliver better results in less time

6. Reduced Complexity

- Complex problems are divided into smaller, manageable parts
- Each part is developed and solved independently, making the system easier to build and maintain

7. Project Management and Collaboration

- Helps in planning, scheduling, and resource management
- Ensures smooth communication and teamwork.

1 .Why Is Software Engineering Needed For Large Projects?

A. To avoid documentation
B. To handle big projects systematically
C. To remove testing
D. To increase coding errors

2.Which of the following is a key benefit of software engineering?

A. Increased complexity
B. Higher development cost
C. Reliable and high-quality software
D. No need for maintenance

3.How does software engineering reduce complexity?

A. By avoiding planning
B. By combining all problems into one
C. By breaking large problems into smaller tasks
D. By eliminating testing.

4.Which activity in software engineering ensures software is free from defects?

A. Coding only
B. Documentation
C. Testing and Quality Assurance

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software. It ensures a systematic workflow and helps align software development with business goals and user requirements.

- Provides a clear and organized framework for managing development phases
- Helps in early detection of defects, reducing overall cost and time
- Ensures high-quality software delivery that meets user expectations

Stages of the Software Development Life Cycle

The Software Development Life Cycle (SDLC) typically consists of six or seven stages, depending on the development model used.

Stage 1: Planning & Feasibility Analysis

This stage determines whether the project is technically, financially, and operationally feasible.

- **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning
- **Output:** Project Plan, Feasibility Report
- **Key Roles:** Project Managers, Senior Engineers, Stakeholders

Stage 2: Requirement Specification (SRS)

In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.

- **Activities:** Requirement gathering, validation, documentation
- **Output:** Software Requirement Specification (SRS)
- **Key Roles:** Business Analysts, Product Owners.

Stage 3: System Design

In this stage, the approved requirements are transformed into a technical blueprint for implementation.

- **High-Level Design (HLD):** Defines system architecture, technology stack, database design, and major modules.
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.
- **Output:** Design Document Specification (DDS)

Stage 4: Development (Coding)

Developers build the software based on the approved design.

- **Activities:** Coding, code reviews, unit testing, version control management
- **Tools:** IDEs, version control systems, debuggers
- **Output:** Source code, executable application
- **Key Roles:** Frontend, Backend, Full Stack Developers

Stage 5: Testing

Testing ensures the software meets requirements and is free from defects before release.

Types of Testing include:

- **Unit Testing**: Verifies individual components
- **Integration Testing**: Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT)**: Confirms business requirements are met

Output: Test cases, defect reports, and quality metrics.

Stage 6: Deployment

The tested software is released to users.

- **Activities:** Production setup, deployment, smoke testing
- **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases
- **Output:** Live application
- **Key Roles:** DevOps Engineers, Release Managers

Stage 7: Maintenance

Post-deployment support ensures long-term usability.

- **Activities:** Bug fixes, performance tuning, updates, feature enhancements
- **Output:** Patches, updates, new versions
- **Key Roles:** Support Engineers, Developers

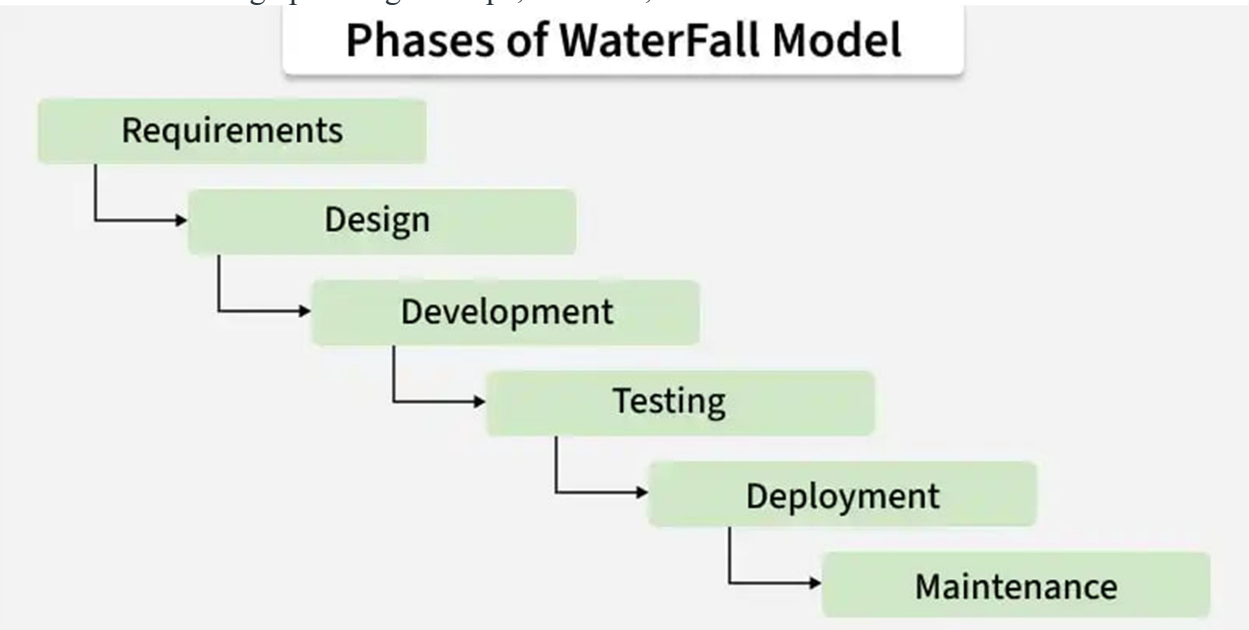
Software Development Life Cycle Models

Software Development Models are structured frameworks that guide the planning, execution, and delivery of software projects. They define the sequence of development stages, such as requirements, design, coding, testing, and deployment.

1.Waterfall Model - Software Engineering

Waterfall Model is a traditional software development approach that follows a linear, step-by-step process where each phase is completed before moving to the next. Its features are:

- Follows a sequential approach where each phase is completed before the next begins.
- Emphasises detailed documentation at every stage of development.
- Ensures quality through proper testing and verification in each phase.
- Involves thorough planning of scope, schedule, and deliverables.



Phases of Waterfall Model

Classical Waterfall Model divides the life cycle into a set of phases. The development process can be considered as a sequential flow in the waterfall. The different sequential phases of the classical waterfall model are as follows:

1. Requirements Analysis and Specification

The first stage focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:** Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.
- **Requirement Specification:** The approved requirements are documented in the Software Requirement Specification (SRS), which serves as a formal agreement between the customer and the development team.

2. Design

The requirements specified in the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):** Defines the overall system architecture, major components, and their interactions.
- **Low-Level Design (LLD):** Provides detailed design of each component, including logic and data flow, to guide developers during coding.
- All designs are documented in the Software Design Document (SDD).

3. Development

This phase involves converting the design into actual working software.

- Developers write source code based on the design documents.
- Suitable programming languages and tools are used.
- Unit testing is performed to verify that each module works correctly on its own.

4. Testing and Deployment

It is ensured that the integrated software functions correctly and is successfully delivered for real world use.

Testing: After unit testing, software modules are integrated and integration testing is performed. Once integration is complete, system testing is carried out to verify the entire system.

- **Alpha Testing:** Performed by the development team.
- **Beta Testing:** Performed by selected end users.

- **Acceptance Testing:** Performed by the customer to approve the software.

Deployment: After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

5. Maintenance

Maintenance ensures the software continues to function effectively after deployment.

- **Corrective Maintenance:** Fixes errors found after release.
- **Perfective Maintenance:** Enhances features based on user needs.
- **Adaptive Maintenance:** Adapts software to new environments.
- **Preventive Maintenance:** Prevents future issues through improvements.

Use Cases

- **Well-Defined Requirements:** Requirements are clear, stable, and fully documented before development begins.
- **Minimal Changes Expected:** The project scope is unlikely to change during development.
- **Small to Medium-Sized Projects:** Suitable for projects with limited complexity and a clear development path.
- **Predictable and Low Risk:** Risks are known, manageable, and can be addressed early in the development cycle.
- **Strict Regulatory Compliance:** Ideal when detailed documentation and adherence to standards or regulations are mandatory.
- **Client Prefers a Sequential Approach:** Best when the client wants a step-by-step, linear development process.
- **Limited Resources:** Works well when resources are limited and need careful planning and allocation.

Example of Waterfall Model

A real-world example of the Waterfall Model using an Online Food Delivery System.

1. **Analysis:** Customer and business requirements are gathered. This includes understanding features such as user registration, restaurant listing, menu display, order placement, payment methods, delivery tracking, and customer support.
2. **Design:** Based on the requirements, the system design is created. This includes deciding the app architecture, database design, user interface layout, payment gateway integration, and security measures. The goal is to ensure smooth ordering, fast performance, and secure transactions.
3. **Development**: Modules such as login, restaurant search, order processing, payment, and notifications are developed. Each feature is built exactly as planned in earlier phases.
4. **Testing:** Once development is complete, the system is tested to ensure everything works correctly. This includes testing order placement, payment processing, delivery tracking, and app performance. Bugs and errors are identified and fixed before release.

5. **Maintenance:** After deployment, the app is maintained to fix issues, improve performance, and add minor updates. This may include adding new restaurants, improving delivery tracking, updating security features, or supporting new payment methods.

Advantages

- Easy to understand and simple to follow.
- Follows a clear step-by-step development process.
- Well-defined stages reduce confusion.
- Clear milestones make progress tracking easier.
- Strong documentation throughout the project.
- Encourages disciplined development practices.
- Suitable for small projects with stable requirements.

1.Which best describes the Waterfall Model?

A. Iterative and flexible
B. Random development process
C. Linear and sequential development
D. Prototype-based development

2.Which phase of the Waterfall Model focuses on understanding and documenting user needs?

A. Design
B. Development
C. Testing
D. Requirements Analysis

3.Which document is created during the Design phase?

A. SRS
B. SDD
C. Test Plan
D. User Manual

4.Which testing is performed by end users before final acceptance?

A. Alpha Testing
B. Unit Testing
C. Beta Testing
D. System Testing

5.The Waterfall Model is best suited for which type of project?

A. Frequently changing requirements
B. Large and highly dynamic systems
C. Small projects with stable requirements

D. Rapid prototyping projects

2 .Iterative Waterfall Model

The Iterative Waterfall Model is an enhanced version of the Classical Waterfall Model that introduces feedback loops between phases, allowing earlier stages to be revisited when issues or changes are identified.

- Allows errors to be identified and corrected early.
- Supports changes by revisiting previous phases.
- Reduces the cost and effort of correcting defects through early feedback.

Phases of Iterative Waterfall Model

1. Requirements Gathering

- Business owners and developers discuss project goals
- Functional and non-functional requirements are collected
- Requirements are clearly documented

2. Design

- A preliminary system design is created
- Architecture, database, and interface design are planned
- Design can be revised based on feedback

3. Implementation

- Developers write the code based on the design
- The system is implemented based on the approved design.
- Errors found can send feedback to the design phase

4. Testing

- The software is tested to ensure it meets requirements
- Bugs and issues are identified early
- Necessary corrections are made through feedback loops

5. Deployment

- The software is deployed and made available to users
- The completed software is deployed for use by end users

6. Review and Improvement

- System performance is reviewed
- Improvements and refinements are made

When to use Iterative Waterfall Model?

- Requirements are mostly clear but may need refinement
- Team is learning new technologies
- Project has a high risk of future changes
- Large projects with limited resources
- When early error detection is important

Application

- Projects with stable core requirements
- Enterprise applications with well-defined requirements
- Large projects divided into manageable phases
- High-risk projects requiring regular validation

Advantages

- Early detection and correction of errors
- Improved collaboration between developers and clients
- Better flexibility than classical Waterfall
- Regular testing improves quality
- Improved product quality through iterative feedback.
- Reduced project risk

Disadvantages

- Changes after development starts are difficult
- No partial or incremental delivery
- Phases cannot overlap
- No formal risk management mechanism
- Customer involvement is limited compared to Agile models.

1.Why was the Iterative Waterfall Model introduced?

A. To remove documentation
B. To allow changes and feedback between phases
C. To eliminate testing
D. To increase development time

3.Which feature differentiates the Iterative Waterfall Model from the Classical Waterfall Model?

A. Parallel development
B. Feedback loops between phases
C. No documentation
D. Rapid prototyping

4.In which phase are bugs mainly identified in the Iterative Waterfall Model?

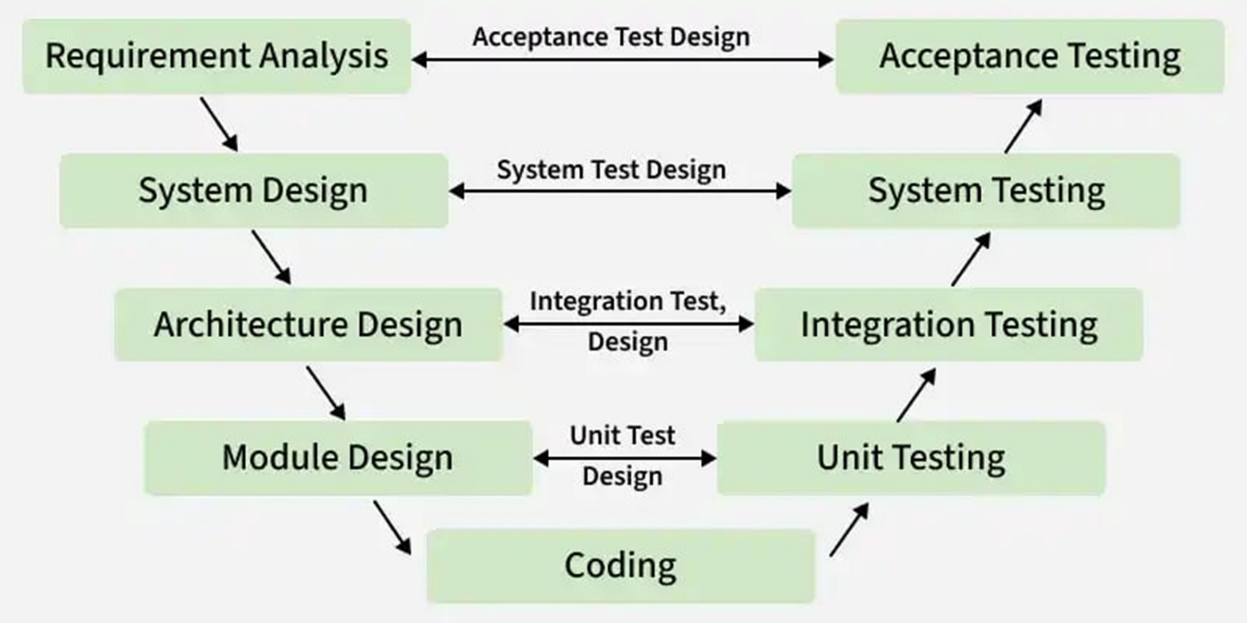
A. Requirements Gathering
B. Design
C. Testing
D. Deployment

5.The Iterative Waterfall Model is best suited for which type of project?

A. Projects with constantly changing requirements
B. Small projects with no risk
C. Large projects with mostly clear requirements
D. Projects needing rapid incremental delivery

3.SDLC V-Model - Software Engineering

The V-Model is a Software Development Life Cycle (SDLC) approach that integrates testing and validation at every stage. It follows a "V" shape structure, where each phase of development is directly connected to a corresponding testing phase, ensuring better quality and early detection of defects.



Phases of SDLC V-Model
The V-Model, which includes the Verification and Validation it is a structural approach to software development. The following are the different Phases of the V-Model of the SDLC.

1. Verification Phases

This is where the process begins. The first step is to gather and understand the customer's needs for the software. The goal is to define the scope of the project clearly to make sure everyone is on the same page.

- Static analysis is performed without executing the code.
- Verifies whether the product meets the specified requirements.

There are several Verification phases in the V-Model:

I. Business Requirement Analysis
- Gather and understand customer requirements.
- Prepare Acceptance Test Plans.

II. System Design
- Create high-level and detailed system design.
- Plan System Testing.

III. Architectural Design
- Define system architecture and module interactions.
- Prepare Integration Test Cases.

IV. Module Design (LLD)
- Design internal details of each module.
- Prepare Unit Test Cases.

V. Coding
- Develop the software based on the design.

- Follow coding standards and perform code reviews.

2. Validation Phases

It involves dynamic analysis techniques (functional, and non-functional), and testing done by executing code. Validation is the process of evaluating the software after the completion of the development phase to determine whether the software meets the customer's expectations and requirements.

I. Unit testing: in **unit testing**, unit test plans are executed to eliminate bugs in code or unit level.
Ii. Integration testing: after completion of unit testing integration testing is performed. In integration testing, the modules are integrated and the system is tested.
3. System testing: **system testing** checks the whole application to see if everything works properly together.
4. User acceptance testing (uat): **user acceptance testing (uat)** is performed in a user environment that resembles the production environment.

Industrial challenge

Modern software projects are more complex and fast-paced, which makes strict requirement definition and early testing even more critical for the v-model.

- Accurately define and refine user requirements.
- Design and build an application according to the authorized user requirements.
- Validate that the application they had built adhered to the authorized business requirements.

Importance

The v-model is an important part of the sdlc, and the process is structured and sequential throughout all the testing.

- **Early defect detection:** finds and fixes defects early, reducing cost and effort.
- **Mapped testing:** each development phase has a corresponding testing phase.
- **Avoids big bang testing:** testing is performed throughout development, not just at the end.
- **Better collaboration:** improves coordination between development and testing teams.
- **Higher quality:** ensures thorough verification and validation for reliable software.

Principles

- **Large to small:** in v-model, testing is done in a hierarchical perspective. As each of these phases is completed the requirements, they are defining become more and more refined and detailed.
- **Data/process integrity:** project design must integrate data and processes cohesively at every stage. Process elements must be identified at every requirement.
- **Scalability:** this principle states that the v-model concept has the flexibility to accommodate any it project irrespective of its size, complexity, or duration.
- **Cross referencing:** a direct correlation between requirements and corresponding testing activity is known as cross-referencing.

Applications

- **Healthcare Systems:** Used in hospital management systems, medical imaging software, and patient monitoring systems where software errors can affect patient safety.
- **Banking and Financial Software:** Applied in internet banking, ATM software, and payment processing systems that require high security and accuracy.
- **Aerospace and Aviation:** Used in flight control systems, navigation software, and aircraft monitoring systems where failures can have serious consequences.

- **Automotive Industry:** Used for developing software in Anti-lock Braking Systems (ABS), airbags, Advanced Driver Assistance Systems (ADAS), and engine control units.

Advantages

- Follows a disciplined, phase-by-phase development process.
- Best for small projects with clear and stable requirements.
- Emphasizes early verification and validation to improve quality.
- Provides a clear, structured process with strong focus on testing.

Disadvantages

- Difficult to accommodate changing requirements.
- Time-consuming due to extensive documentation and testing.
- Not suitable for complex or high-risk projects.
- Does not support iterative development.

1 .In the V-Model of SDLC, what testing activity directly corresponds to the requirements analysis phase?

A. Acceptance Testing
B. Unit Testing
C. Integration Testing
D. System Testing

2. In the V-Model, why are test plans prepared during the corresponding development phases rather than after coding is complete?

A. To eliminate the coding phase
B. To detect defects early and validate requirements
C. To allow requirements to change freely
D. To avoid performing system testing

3 . A team is developing software where requirements are clearly defined and unlikely to change. Which SDLC model would be most appropriate if the team wants testing to be planned alongside development?

A. Agile Model
B. Spiral Model
C. V-Model
D. RAD Model

4.A requirement states that an online banking system must allow users to transfer money securely. Which testing phase primarily validates whether the completed system satisfies this business requirement from the user's perspective?

A. Unit Testing
B. Integration Testing
C. System Testing
D. User Acceptance Testing

5 A project manager chooses the V-Model mainly because the project requires a disciplined development process and rigorous testing at every level. Which statement best describes the model?

A. Testing is postponed until the entire product is developed
B. Testing activities are mapped to corresponding development activities
C. Requirements are continuously changed throughout development
D. Development and testing occur without any defined relationship

4.Incremental Process Model - Software Engineering

The Incremental Process Model is a software development approach where the system is built and delivered in small, manageable parts called increments. Each increment adds new features until the complete system is developed.

- Develops the system in small, manageable increments.
- Allows changes and new features in later increments.
- Combines structured planning with iterative development.



Phases of Incremental Model

The Incremental Model is divided into multiple phases, where each increment goes through a complete development cycle to build the system step by step.

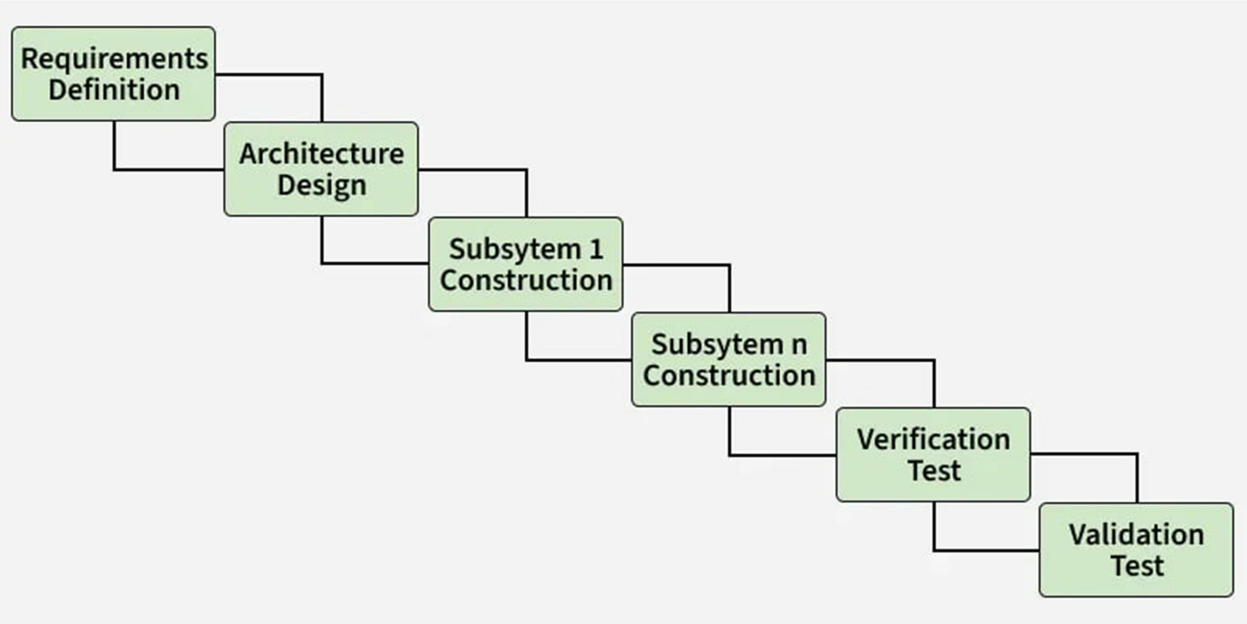
1. **Requirement Analysis:** Requirements for the current increment are gathered and clearly defined.
2. **Design & Development:** The system design is prepared, and new features are developed.
3. **Testing:** Each increment is tested to ensure new and existing features work correctly.

4. **Implementation:** The tested increment is delivered and integrated with previous versions. These steps repeat until the final system is completed.

Types of Incremental Model
The Incremental Model has two main types, each offers different approaches to how software is developed in parts.

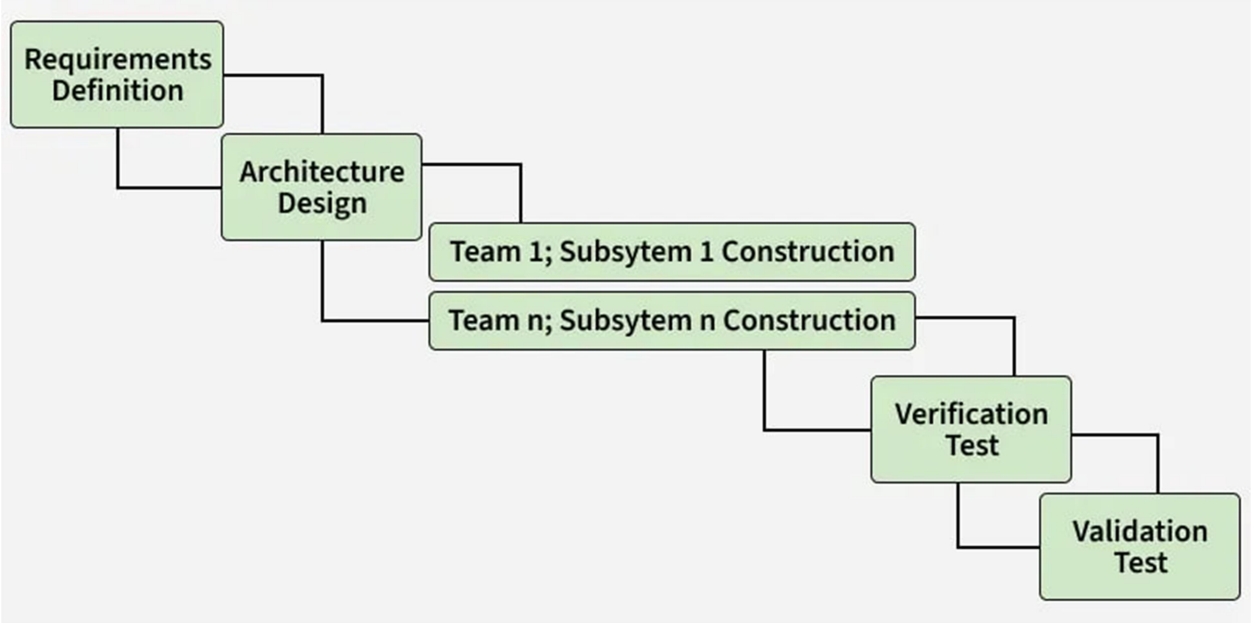
1. Staged Delivery Model

Develops software in a sequence of planned stages, where each stage delivers a functional part of the system. Each release brings the product closer to completion, allowing it to evolve gradually. Working versions are delivered at regular intervals, making progress visible and manageable throughout the development process.



2. Parallel Development Model

Divides the system into multiple modules that are developed in parallel by different teams. By working on separate components in parallel, the development process becomes faster and more efficient. This approach reduces overall project time and allows teams to focus on specific functionalities concurrently.



Advantages

- Early Software Delivery, improves customer satisfaction
- Easy to Manage Changes between increments
- Better Risk Management through early testing
- Cost Control by spreading budget over stages
- Simpler Debugging due to modular development
- High Software Quality through continuous testing

Note: *The Incremental Model is not ideal when the software development team is not highly skilled or experienced because managing increments and integrating them well requires a competent team.*

Disadvantages

- Requires Skilled Team and proper planning
- Cost May Increase due to repeated testing and integration
- Design Issues if future requirements are unclear
- Integration Challenges between increments
- Repeated Bug Fixing across multiple increments

Suggested Quiz

1.What is the main idea of the Incremental Process Model?

A. Delivering software only at the end
B. Developing and delivering software in small parts
C. Avoiding customer feedback
D. Completing all phases simultaneously

2.Which is a key advantage of the Incremental Process Model?

A. No need for testing
B. Early delivery of working software
C. Fixed and rigid requirements
D. No integration required

3.Which type of Incremental Model develops different modules simultaneously?

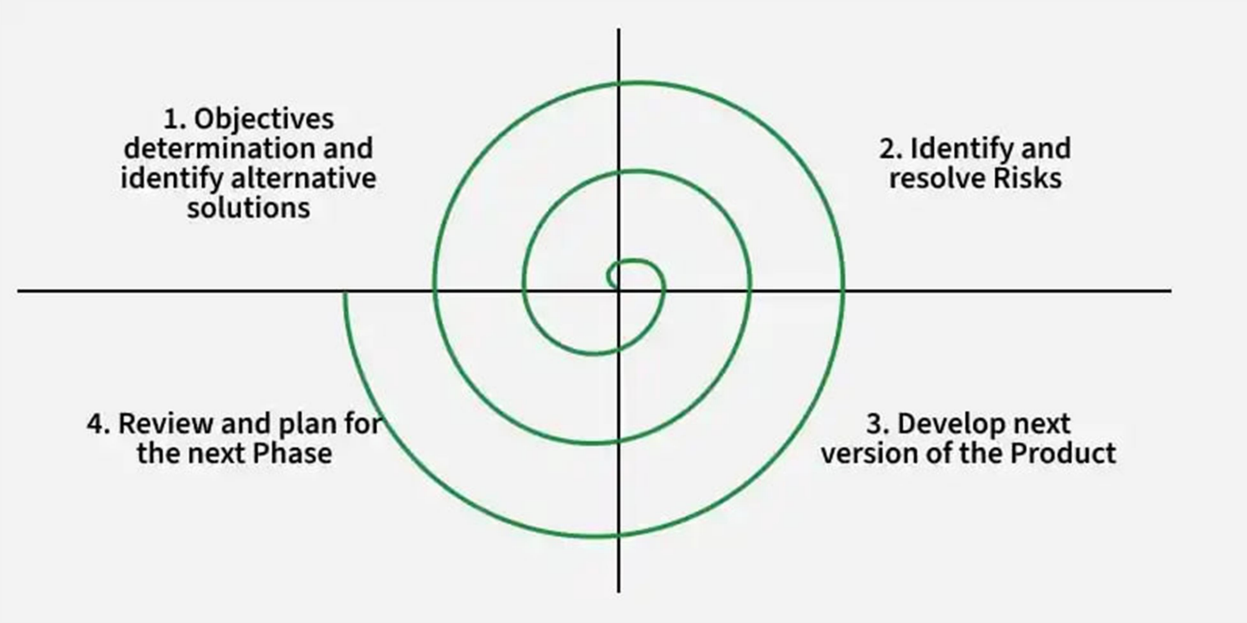
A. Staged Delivery Model
B. Classical Waterfall Model
C . Parallel Development Model
D . Prototype Model

4.The Incremental Process Model is most suitable when:

A. Requirements are unclear and constantly changing
B. No skilled developers are available
C. Early release of important features is required
D. No customer feedback is needed

5.Spiral Model in Software Engineering

The spiral model is a risk-driven software development process model. Based on the unique risk patterns of a given project, the spiral model guides a team to adopt elements of one or more process models, such as incremental, waterfall, or evolutionary prototyping.



Phases of the Spiral Model

Focus is on managing risk through multiple iterations of the software development process. Each phase of the Spiral Model is divided into four Quadrants.

1. Objective Setting (Planning)

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored

2. Risk Analysis

- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

3. Development and Testing

- Selected features are designed, developed, and tested
- The next working version of the software is created

4. Review and Planning

- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation. The Radius of the spiral at any point represents the expenses (cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.

Risk Handling

- Risks are identified and analyzed at the beginning
- Prototypes help detect and resolve risks early
- Continuous risk evaluation at every iteration
- More flexible than Prototyping and Waterfall models

The Spiral Model is often called a Meta-Model because it combines concepts from multiple SDLC models such as Waterfall, Iterative, and Prototyping models.

Example

Following shows how the Spiral Model supports continuous improvement and risk management at every stage in building E Commerce Website.

- **Requirements & Prototype:** Gather basic requirements like product listing, cart, and payment, identify risks, and build a prototype.
- **Design & Development:** Refine the design using feedback and develop core features such as secure payments, user registration, and shopping cart functionality, then test and refine them.
- **Advanced Features:** Add advanced features such as order tracking, product reviews, and search functionality are developed, and also test performance, security, and scalability.
- **Deployment & Maintenance:** The system is fully tested, deployed for users, and continuously monitored to address remaining risks and make further improvements.

Advantages

- **Effective Risk Handling:** Identifies and resolves risks at every phase, making it ideal for projects with high uncertainty.
- **Suitable for Large Projects:** Best suited for large and complex software systems.
- **Flexible Requirements:** Allows changes in requirements even at later stages of development.
- **Early Customer Feedback:** Customers can review early versions of the product, improving satisfaction.
- **Iterative Development:** Software is developed in repeated cycles, allowing gradual improvement.
- **Better Communication:** Regular reviews improve coordination between customers and developers.
- **High Software Quality:** Continuous testing and refinement lead to reliable and high-quality software.

Disadvantages

- **Complex to Manage:** The model is complex due to multiple iterations and continuous risk analysis.
- **High Cost:** Not suitable for small projects because it requires more time, effort, and resources.
- **Depends Heavily on Risk Analysis:** Success depends on accurate risk identification, which requires experienced professionals.
- **Difficult Time Estimation:** The number of iterations is not fixed, making scheduling difficult.
- **Time-Consuming:** Frequent reviews and evaluations can increase development time.
- **Resource Intensive:** Requires significant investment in planning, risk management, and testing.

Suggested Quiz

1 .The Spiral Model is mainly used for:

A. Small projects with fixed requirements
B. Rapid coding without planning
C. Risk handling in large and complex projects
D. Projects with no customer involvement

2. Each loop of the Spiral Model represents:

A. A testing phase only
B. A complete development cycle
C. A documentation phase
D. A maintenance activity

3 .Why is the Spiral Model called a Meta-Model?

A. It is used only for testing
B. It supports only small projects
C. It combines features of multiple SDLC models
D. It eliminates documentation

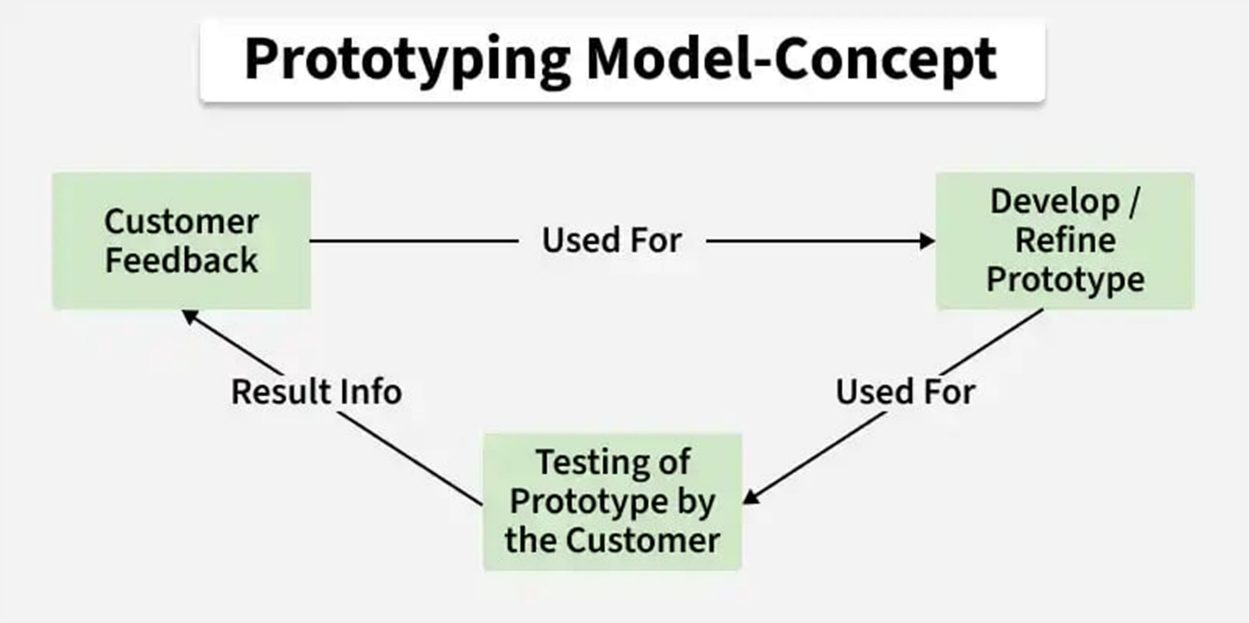
4. Which phase of the Spiral Model focuses on identifying and resolving risks?

A. Objectives Definition
B. Development and Testing
C. Review and Planning
D. Risk Analysis and Resolution

Prototyping Model

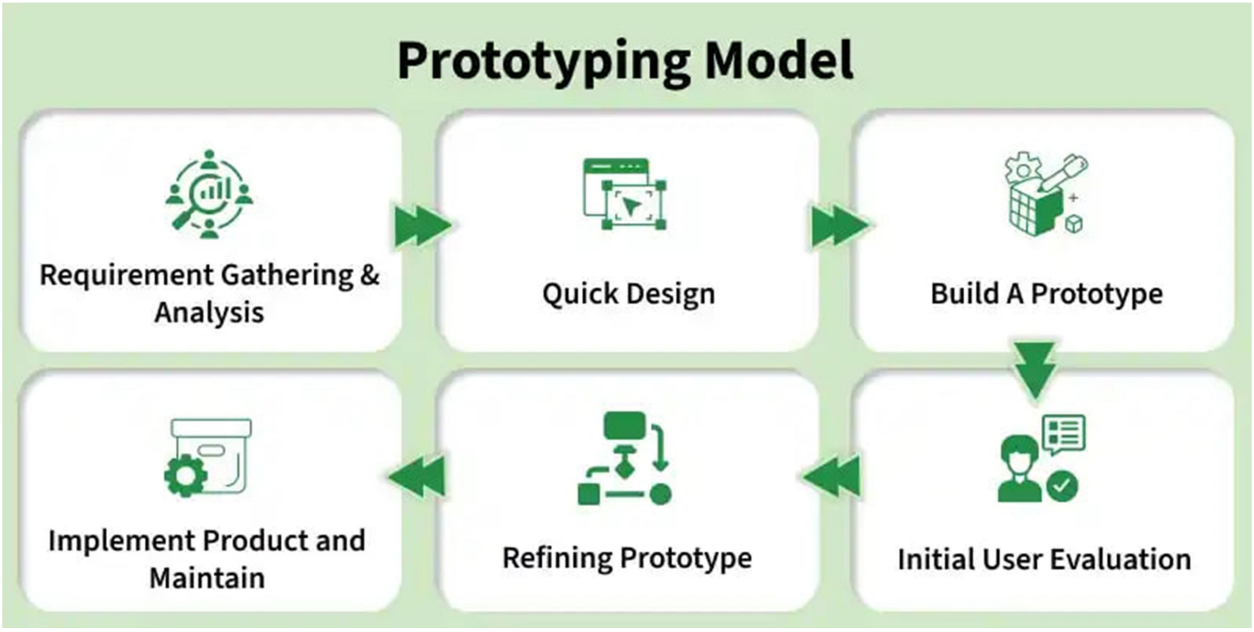
The Prototyping Model is a widely used Software Development Life Cycle Model (SDLC Model) in which an early version of the software, called a prototype, is developed and shared with users for feedback.

- Early prototypes allow users to interact with the system and identify missing functionalities or issues.
- Iterative development ensures the final product is refined, efficient, and meets customer requirements.
- Useful when project requirements are unclear or likely to change, as prototypes can be iteratively refined based on user input.
- Helps create a final product that closely aligns with user expectations and reduces the risk of errors.



Phases of Prototyping Model

Prototyping Model has six phases as follows:



1. Requirements gathering and analysis: Requirement analysis is the first step in developing a prototyping model. During this phase, the system requirements are defined. Users are interviewed to understand their expectations and requirements.
2. Quick design: The second phase could consist of a preliminary design or a quick design. During this stage, the system's basic design is formed. However, it is not a complete design. It provides the user with a quick overview of the system. The rapid design aids in the development of the prototype.
3. Build a Prototype: During this stage, an actual prototype is intended to support the knowledge gained from quick design. It is a small low-level working model of the desired system.
4. Initial user evaluation: The proposed system is presented to the client for preliminary testing at this stage. It is beneficial to investigate the performance model's strengths and weaknesses. Customer feedback and suggestions are gathered and forwarded to the developer.
5. Refining prototype: If the user is dissatisfied with the current model, you may want to improve the type that responds to user feedback and suggestions. When the user is satisfied with the upgraded model, a final system based on the approved final type is created.
6. Implement Product and Maintain: The final system was fully tested and distributed to production after it was developed to support the original version. To reduce downtime and prevent major failures, the programmer is run on a regular basis.

Types of Prototyping Models

There are four types of Prototyping Models, which are described below.



Prototyping validating design decisions and guiding development it

1. Rapid Throwaway Prototyping: This technique offers a useful method of exploring ideas and getting customer feedback for each of them.

- In this method, a developed prototype need not necessarily be a part of the accepted prototype.
- Customer feedback helps prevent unnecessary design faults and hence, the final prototype developed is of better quality.

2. Evolutionary Prototyping: In this method, the prototype developed initially is incrementally refined based on customer feedback till it finally gets accepted.

- In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort.
- This saves time and effort because developers do not need to build a new prototype from scratch for every iteration.

3. Incremental Prototyping: In this type of incremental prototyping, the final expected product is broken into different small pieces of prototypes and developed individually.

- In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order.
- It is efficient approach that reduces the complexity of the development process, where the goal is divided into sub-parts and each sub-part is developed individually.
- The time interval between the project's beginning and final delivery is substantially reduced because all parts of the system are prototyped and tested simultaneously.
- However, integration issues may occur if the interfaces and overall system architecture are not carefully planned before development.

4. Extreme Prototyping: This method is mainly used for web development. It consists of three sequential independent phases:

- In this phase, a basic prototype with all the existing static pages is presented in HTML format.
- In the 2nd phase, Functional screens are made with a simulated data process using a prototype services layer.
- This is the final step where all the services are implemented and associated with the final prototype.

This Extreme Prototyping method makes the project cycling and delivery robust and fast and keeps the entire developer team focused and centralized on product deliveries rather than discovering all possible needs and specifications and adding necessitated features.

Applications

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.

Advantages

- Customers can see early versions of the product, improving satisfaction and comfort.
- New requirements and missing functionalities can be easily identified and incorporated.
- Early error detection saves effort, cost, and enhances software quality.
- Prototypes can be reused for future projects and offer flexibility in design.
- Facilitates communication, collaboration, and alignment between technical and non-technical stakeholders.
- Reduces project risk b eratively.

Disadvantages

- Can be time-consuming and costly due to repeated iterations.
- Frequent changes in requirements may lead to poor documentation and development challenges.
- Prototypes may not fully represent the final product, causing scalability or functionality issues.
- Risk of sub-optimal solutions if developers rush to build prototypes.
- Focus on prototyping can delay the final product or give a false sense of completion.
- Customers may demand early delivery or lose interest if initial prototypes are unsatisfactory.

Classification of Software Requirements - Software Engineering

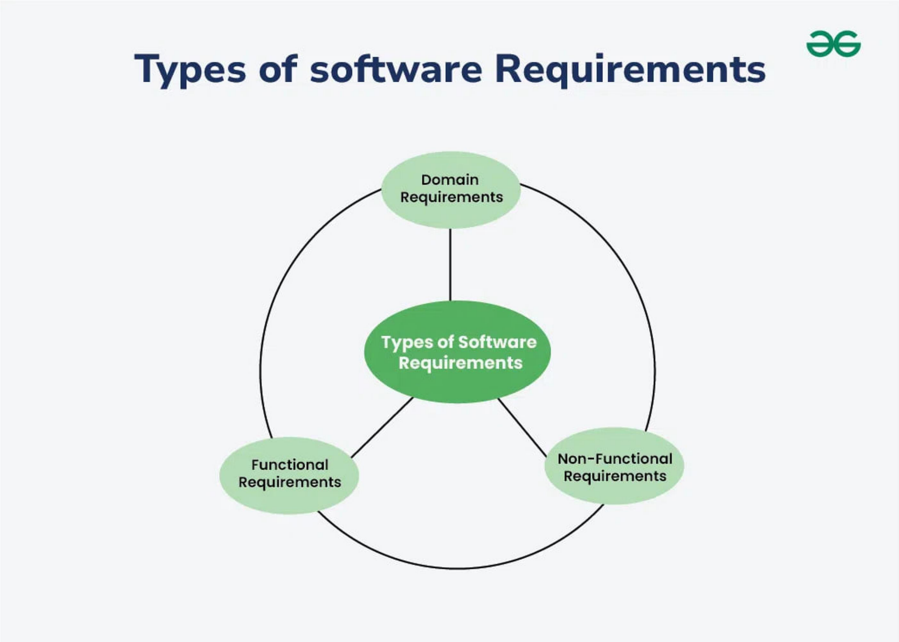
Classification of Software Requirements is important in the software development process. It organizes our requirements into different categories that make them easier to manage, prioritize, and track. The main types of Software Requirements are functional, non-functional, and domain requirements.

What is Software Requirements?
According to IEEE standard 729, a requirement is defined as follows:

- A condition or capability needed by a user to solve a problem or achieve an objective
- A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification or other formally imposed documents
- A documented representation of a condition or capability, as in 1 and 2.

Types of Software Requirements
Software Requirements are mainly classified into three types:

- Functional requirements
- Non-functional requirements
- Domain requirements



1. Functional Requirements

Definition: Functional requirements describe what the software should do. They define the functions or features that the system must have.
Examples:

- **User Authentication**: The system must allow users to log in using a username and password.
- **Search Functionality**: The software should enable users to search for products by name or category.
- **Report Generation**: The system should be able to generate sales reports for a specified date range.

Explanation: Functional requirements specify the actions that the software needs to perform. These are the basic features and functionalities that users expect from the software.

2. Non-functional Requirements

Definition: Non-functional requirements describe how the software performs a task rather than what it should do. They define the quality attributes, performance criteria, and constraints.
Examples:

- **Performance**: The system should process 1,000 transactions per second.
- **Usability**: The software should be easy to use and have a user-friendly interface.
- **Reliability**: The system must have 99.9% uptime.
- **Security**: Data must be encrypted during transmission and storage.

Explanation: Non-functional requirements are about the system's behavior, quality, and constraints. They ensure that the software meets certain standards of performance, usability, reliability, and security.

3. Domain Requirements

Definition: Domain requirements are specific to the domain or industry in which the software operates. They include terminology, rules, and standards relevant to that particular domain.
Examples:

- **Healthcare**: The software must comply with HIPAA regulations for handling patient data.
- **Finance**: The system should adhere to GAAP standards for financial reporting.
- **E-commerce**: The software should support various payment gateways like PayPal, Stripe, and credit cards.

Explanation: Domain requirements reflect the unique needs and constraints of a particular industry. They ensure that the software is relevant and compliant with industry-specific regulations and standards.

What are Functional Requirements?

Functional Requirements are the requirements that the end user specifically demands as basic facilities that the system should offer. It can be a calculation, data manipulation, business process, user interaction, or any other specific functionality that defines what function a system is likely to perform. All these functionalities need to be necessarily incorporated into the system as a part of the contract. These are represented or stated in the form of input to be given to the system, the operation performed and the output expected.

- They are the requirements stated by the user which one can see directly in the final product, unlike the non-functional requirements. For example, in a hospital management system, a doctor should be able to retrieve the information of his patients.
- Each high-level functional requirement may involve several interactions or dialogues between the system and the outside world.
- To accurately describe the functional requirements, all scenarios must be enumerated.
- There are many ways of expressing functional requirements e.g., natural language, a structured or formatted language with no rigorous syntax, and formal specification language with proper syntax.
- Functional Requirements in Software Engineering are also called Functional Specification.

What are Non-functional Requirements?

These are basically the quality constraints that the system must satisfy according to the project contract. Nonfunctional requirements, not related to the system functionality, rather define how the system should perform The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements. They basically deal with issues like:

- **Portability**
- **Security**
- **Maintainability**
- **Reliability**

- **Scalability**
- **Performance**
- **Reusability**
- **Flexibility**

Non-functional requirements are classified into the following types:
- Interface constraints
- Performance constraints: response time, security, storage space, etc.
- Operating constraints
- Life cycle constraints: maintainability, portability, etc.
- Economic constraints

The process of specifying non-functional requirements requires the knowledge of the functionality of the system, as well as the knowledge of the context within which the system will operate.

They are divided into two main categories
- **Execution qualities:** These consist of thing like security and usability, which are observable at run time.
- **Evolution qualities:** These consist of things like testability, maintainability, extensibility, and scalability that are embodied in the static structure of the software system.

What are Domain requirements?
Domain requirements are the requirements that are characteristic of a particular category or domain of projects. Domain requirements can be functional or nonfunctional. Domain requirements engineering is a continuous process of proactively defining the requirements for all foreseeable applications to be developed in the software product line. The basic functions that a system of a specific domain must necessarily exhibit come under this category. For instance, in academic software that maintains records of a school or college, the functionality of being able to access the list of faculty and list of students of each grade is a domain requirement. These requirements are therefore identified from that domain model and are not user-specific.

Classifications of Software Requirements

Other common classifications of software requirements can be:

1. **User requirements:** These requirements describe what the end-user wants from the software system. User requirements are usually expressed in natural language and are typically gathered through interviews, surveys, or user feedback.
2. **System requirements:** These requirements specify the technical characteristics of the software system, such as its architecture, hardware requirements, software components, and interfaces. System requirements are typically expressed in technical terms and are often used as a basis for system design.
3. **Business requirements**: These requirements describe the business goals and objectives that the software system is expected to achieve. Business requirements are usually expressed in terms of revenue, market share, customer satisfaction, or other business metrics.
4. **Regulatory requirements:** These requirements specify the legal or regulatory standards that the software system must meet. Regulatory requirements may include data privacy, security, accessibility, or other legal compliance requirements.

5. **Interface requirements**: These requirements specify the interactions between the software system and external systems or components, such as databases, web services, or other software applications.
6. **Design requirements:** These requirements describe the technical design of the software system. They include information about the software architecture, data structures, algorithms, and other technical aspects of the software.

By classifying software requirements, it becomes easier to manage, prioritize, and document them effectively. It also helps ensure that all important aspects of the system are considered during the development process.

Advantages of Classifying Software Requirements

Advantages of classifying software requirements include:

1. **Better organization:** Classifying software requirements helps organize them into groups that are easier to manage, prioritize, and track throughout the development process.
2. **Improved communication:** Clear classification of requirements makes it easier to communicate them to stakeholders, developers, and other team members. It also ensures that everyone is on the same page about what is required.
3. **Increased quality:** By classifying requirements, potential conflicts or gaps can be identified early in the development process. This reduces the risk of errors, omissions, or misunderstandings, leading to higher-quality software.
4. **Improved traceability:** Classifying requirements helps establish traceability, which is essential for demonstrating compliance with regulatory or quality standards.

Disadvantages of classifying software requirements

Disadvantages of classifying software requirements include:

1. **Complexity**: Classifying software requirements can be complex, especially if there are many stakeholders with different needs or requirements. It can also be time-consuming to identify and classify all the requirements.
2. **Rigid structure**: A rigid classification structure may limit the ability to accommodate changes or evolving needs during the development process. It can also lead to a siloed approach that prevents the integration of new ideas or insights.
3. **Misclassification**: Misclassifying requirements can lead to errors or misunderstandings that can be costly to correct later in the development process.

Overall, the advantages of classifying software requirements outweigh the disadvantages, as it helps ensure that the software system meets the needs of all stakeholders and is delivered on time, within budget, and with the required quality.

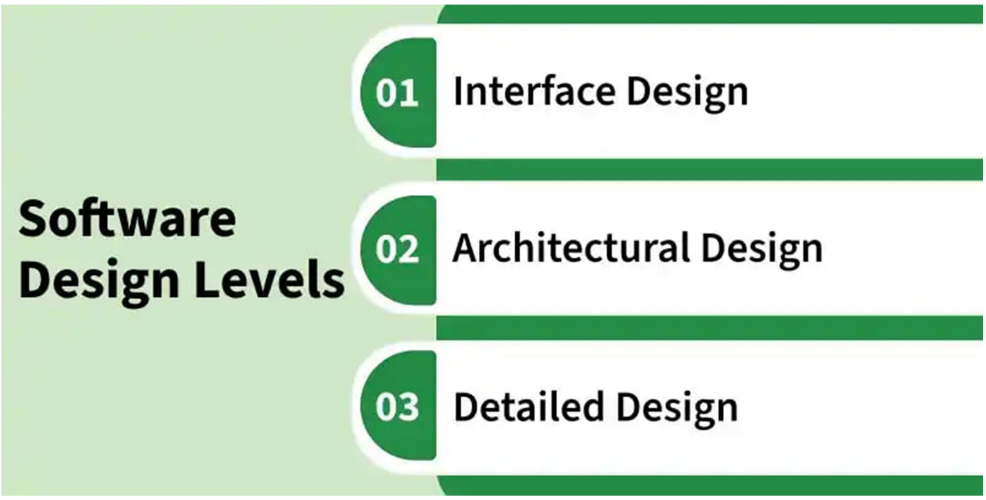
Conclusion

Classifying software requirements provides numerous benefits, such as better organization, improved communication, increased quality, and enhanced traceability. By systematically categorizing the requirements development teams can be sure that the software meets the requirements of stakeholders while observing standards and delivered with efficiency and effectiveness.

Software Design Process

Software Design processes the phase where developers plan how to turn a set of requirements into a working system like a blueprint for the software.

- Break down complex requirements into smaller, manageable modules or components.
- It bridges functional and non-functional requirements to coding, preventing costly rework.
- Design the system architecture and define how different components interact and work together.



Software Design Levels
The software design process can be divided into the following three levels or phases of design:

1. Interface Design

It is the specification of the interaction between a system and its environment. This phase proceeds at a high level of abstraction with respect to the inner workings of the system i.e, during interface design, the internal of the systems are completely ignored, and the system is treated as a black box. Attention is focused on the dialogue between the target system and the users, devices, and other systems with which it interacts. The design problem statement produced during the problem analysis step should identify the people, other systems, and devices which are collectively called agents.

Interface design should include the following details:

- Precise description of events in the environment, or messages from agents to which the system must respond.
- Precise description of the events or messages that the system must produce.
- Specification of the data, and the formats of the data coming into and going out of the system.
- Specification of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

2. Architectural Design

It is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored. Issues in architectural design includes:

- Gross decomposition of the systems into major components.
- Allocation of functional responsibilities to components.
- Component Interfaces.
- Component scaling and performance properties, resource consumption properties, reliability properties, and so forth.
- Communication and interaction between components.

The architectural design adds important details ignored during the interface design. Design of the internals of the major components is ignored until the last phase of the design.

3. Detailed Design

It is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures. The detailed design may include:

- Decomposition of major system components into program units.
- Allocation of functional responsibilities to units.
- User interfaces.
- Unit states and state changes.
- Data and control interaction between units.
- Data packaging and implementation, including issues of scope and visibility of program elements.
- Algorithms and data structures.

Software Design Phases

The Actual Software Design process travel through the one end to another end of making and Software well with completion of requirements and the phases which included in the same that are bellow:

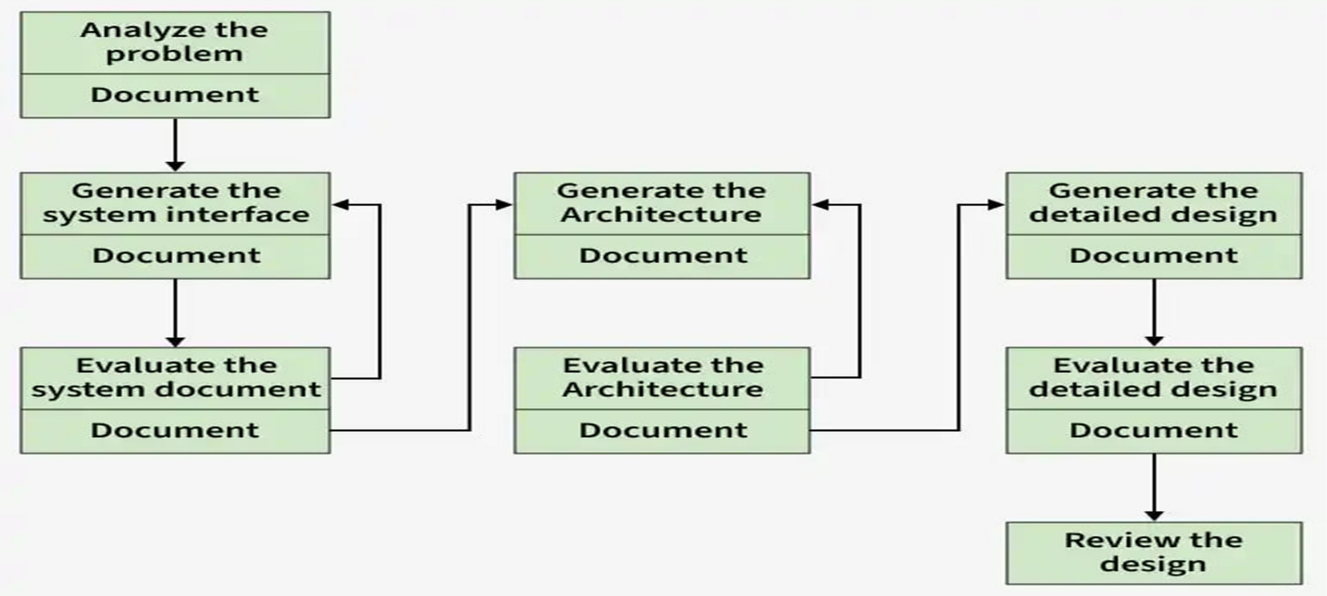


Software Design Phases

1. **Understanding Project Requirements:** Before jumping into the design, the first step is to make sure the team understands what the users need, what the business goals are, and any potential challenges. This understanding sets the foundation for creating a design that meets both user and business needs.
2. **Research, Analysis, and Planning:** During this phase, the team gathers data through methods like interviews, surveys, and focus groups. This helps get a clearer picture of what the users want and allows the team to design with the user in mind, ensuring the software will truly meet their needs.
3. **Designing the Software:** In the design phase, the team starts creating visual elements like wireframes, user stories, and flow diagrams to map out how the system will work. Based on the feedback, prototypes are created and fine-tuned to make sure the design is moving in the right direction.
4. **Technical Design:** After gathering feedback, the team gets into more details with the technical design. This phase involves creating a thorough technical document that outlines exactly how the software will be implemented, including specific components and how they will work together.
5. **User Interface Design:** The focus here is on making sure the software is easy to use. UI designers work on the visual elements, navigation, and overall user experience to ensure the interface is intuitive and friendly.
6. **Prototyping:** Prototypes are created to visualize the design and functionality before the full development process begins. These can range from simple wireframes (low fidelity) to fully interactive models (high fidelity) depending on the stage of development.

Elements of Software Design

Good software design is built on several core element that work together to create effective system.



1. **Architecture:** This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture. A solid architecture verify the system is flexible, stable, and easy to maintain over time.
2. **Modules:** Modules as the building blocks of the system. Each one handles a specific task or feature. Breaking a system into smaller modules makes it easier to develop, test, and maintain the system. These are components that handle one specific task in a system. A combination of the modules makes up the system.
3. **Components:** This provides a particular function or group of related functions. They are made up of modules. Organizing the system into components helps keep the code clean and makes the system more adaptable.
4. **Interfaces:** These are smaller units within modules that focus on specific functions. This is the shared boundary across which the components of a system exchange information and relate.
5. **Data:** Data is at the heart of any system. It's all about how information is stored, accessed, and shared.This is the management of the information and data flow.

How Software Design Fits into the SDLC

Software Design comes when the project requirement are done and when we are about to start the Development process, Here is How Software Design fits into the Software Development Life Cycle.

- Design the System Architecture: Create the overall structure of the software, including modules, components, and their interactions.
- Prepare Design Artifacts: Develop wireframes, UI/UX designs, data flow diagrams (DFDs), and database designs to visualize the system.

- Define Component Functionality: Specify how each component will work together to ensure smooth functionality and efficient data flow.
- Begin Software Development: Once the design is finalized and approved, the implementation (coding) phase starts, where developers build the actual software.

Tools for Software Design
There are many tools that make the process easier and more efficient, whether you're working on wireframes, prototypes, or documentation. Here are some popular tools:

1. Figma
2. Balsamiq
3. Axure RP
4. Sketch
5. InVision Studio

Software Measurement and Metrics

Software measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process.

- Software measurement is the quantitative assessment of a characteristic of a software product or process.
- It is an important part of software engineering, and its process is defined by ISO standards.



Software Measurement Principles

The software measurement process can be characterized by five activities:

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Need for Software Measurement

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.
- Enable data-driven decision-making in project planning and control.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Give software products and processes a quantitative basis for evaluation.
- Enable the ongoing improvement of software development practices.

Classification of Software Measurement

There are 2 types of software measurement:

1. **Direct Measurement:** In direct measurement, the product, process, or thing is measured directly using a standard scale.
2. **Indirect Measurement:** In indirect measurement, the quantity or quality to be measured is measured using related parameters i.e. by use of reference.

Software Metrics

A metric is a measurement of the level at which any impute belongs to a system product or process. Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

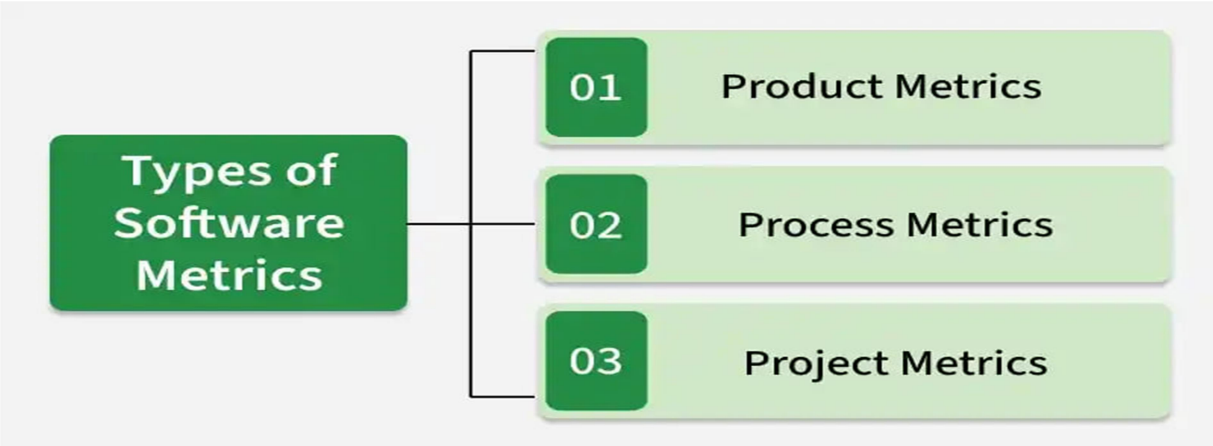
1. Planning
2. Organizing
3. Controlling

4. Improving

Characteristics of Software Metrics

- **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.
- **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.
- **Applicability:** Metrics should be applicable in the initial phases of the development of the software.
- **Repeatable:** When measured repeatedly, the metric values should be the same and consistent.
- **Economical:** The computation of metrics should be economical.
- **Language Independent:** Metrics should not depend on any programming language.

Types of Software Metrics



Product Metrics: Product metrics are used to evaluate the state of the product, tracing risks and undercover prospective problem areas. The ability of the team to control quality is evaluated. Examples include lines of code, cyclomatic complexity, code coverage, defect density, and code maintainability index.

2. Process Metrics: Process metrics pay particular attention to enhancing the long-term process of the team or organization. These metrics are used to optimize the development process and maintenance activities of software. Examples include effort variance, schedule variance, defect injection rate, and lead time.
3. Project Metrics: The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-

- Number of software developer
- Staffing patterns over the life cycle of software

- Cost and schedule
- Productivity

Advantages

- Reduction in cost or budget and managing the workloads and teams.
- Reduction in overall time to produce the product.
- It helps to identify the particular area for improvising and also to increase the product quality.
- It helps to determine the complexity of the code and to test the code with resources.
- It helps in providing effective planning, controlling and managing of the entire product.

Disadvantages

- It is expensive and difficult to implement the metrics in some cases.
- Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
- Sometimes the quality of the product is not met with the expectation.
- It leads to measure the unwanted data which is wastage of time.
- Measuring the incorrect data leads to make wrong decision making.